

**ĐẠI HỌC QUỐC GIA TP. HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH**



BÁO CÁO COURSEWORK

Môn: Học sâu và ứng dụng trong thị giác máy tính

CO5085 - E1–E3, A1 và A2

**Phân loại ảnh, Transformer, mô hình chuỗi,
CNN vs Transformer và Object Detection**

Thực hiện

[CẦN ĐIỀN: HỌ TÊN THÀNH VIÊN 1]

[CẦN ĐIỀN: MSSV 1]

[CẦN ĐIỀN: HỌ TÊN THÀNH VIÊN 2]

[CẦN ĐIỀN: MSSV 2]

Tên nhóm: [CẦN ĐIỀN: TÊN NHÓM]

Giảng viên: Lê Thành Sách, Nguyễn Quốc Minh

Thành phố Hồ Chí Minh, 10/2026

Mục lục

1	Xác định đề bài, phạm vi và mục tiêu	1
1.1	Đối chiếu yêu cầu của đề với cấu trúc báo cáo	1
1.2	Phạm vi thí nghiệm dự kiến	2
1.3	Quy tắc trung thực và sử dụng AI	2
2	Thiết lập chung, tổ chức code và khả năng tái lập	3
2.1	Môi trường tính toán	3
2.2	Quy ước code	3
2.3	Artifact và naming convention	3
2.4	Quy trình notebook	3
3	E1 - Phân loại ảnh: Softmax, MLP và CNN	4
3.1	Mục tiêu, dữ liệu và split	4
3.2	Tiền xử lý và DataLoader	4
3.3	Kiến trúc bốn cấu hình	4
3.4	Training loop và protocol	4
3.5	Kết quả định lượng	5
3.6	Phân tích lỗi và kết luận E1	5
4	E2 - Transformer: Multi-Head Self-Attention và tokenization	6
4.1	Mục tiêu và nguyên tắc hiện thực	6
4.2	Multi-Head Self-Attention tự viết	6
4.3	Ba cách tokenization	6
4.4	Classifier và ma trận thí nghiệm	7
4.5	Kết quả và attention visualization	7
4.6	Thảo luận E2	7
5	E3 - LSTM/GRU trên chuỗi ảnh	8
5.1	Mục tiêu và các cách biến ảnh thành chuỗi	8
5.2	Manual LSTM và GRU	8
5.3	Ma trận thí nghiệm	8
5.4	Kết quả	9
5.5	Thảo luận E3	9
6	A1 - CNN vs Transformer trên tập ảnh lớn	9

6.1	Bài toán, dataset và A1.1 protocol	9
6.2	Split, DataLoader và augmentation	10
6.3	CNN: custom ResNet-18	10
6.4	Transformer: custom ViT/DeiT-Tiny	10
6.5	Protocol scratch, pretrained và freezing	11
6.6	Thiết lập huấn luyện và chi phí	11
6.7	Kết quả so sánh	11
6.8	Thảo luận A1	12
7	A2 - Object Detection và phân tích paper	12
7.1	Bài toán, dataset và A2.1 protocol	12
7.2	Dataset, DataLoader và augmentation	13
7.3	Paper chính: Feature Pyramid Networks for Object Detection	13
7.3.1	Ý tưởng, kiến trúc và giả thiết	13
7.3.2	Thí nghiệm của paper	13
7.3.3	Hạn chế của paper và khả năng tái hiện	14
7.4	Pipeline Faster R-CNN tự ghép theo từng thành phần	14
7.5	Ma trận thí nghiệm A2	14
7.6	Training protocol	15
7.7	Kết quả định lượng	15
7.8	Failure-case analysis	16
7.9	Đối chiếu paper với pipeline của nhóm	16
7.10	Hạn chế và kết luận A2	16
8	Kết luận và hướng mở rộng	16
A	Phụ lục: lệnh chạy và tái lập	17
A.1	Cài đặt	17
A.2	Build báo cáo	17
A.3	Artifact cần giữ để tái lập	17
	Tài liệu tham khảo	18

Danh sách hình vẽ

1	Luồng tổng thể của coursework: dữ liệu → EDA/split → model components → training → evaluation → figures/tables → report.	2
2	EDA Fashion-MNIST: sample grid và phân bố 10 lớp.	4
3	Learning curves của bốn cấu hình E1.	5
4	Confusion matrix và các mẫu dự đoán sai tiêu biểu của E1.	5
5	Minh họa ba cách tokenization trên cùng một ảnh Fashion-MNIST.	6
6	So sánh accuracy và thời gian giữa manual/PyTorch MSA và các tokenizer.	7
7	Attention heatmap của một số mẫu Fashion-MNIST.	7
8	Sơ đồ manual LSTM/GRU cell và luồng ảnh → sequence → classifier.	8
9	Learning curves và so sánh E3 với MLP/CNN baseline.	9
10	EDA A1: class distribution, image-size distribution và sample grid.	10
11	Sơ đồ ResNet-18 và ViT/DeiT-Tiny dùng trong A1.	10
12	Learning curves của các cấu hình A1 tiêu biểu.	12
13	Confusion matrix và error gallery của best CNN/Transformer.	12
14	Accuracy/F1 so với trainable parameters, training time và peak VRAM.	12
15	EDA A2: số box theo lớp, số object/ảnh, phân bố diện tích/aspect ratio và sample annotations.	13
16	Top-down pathway và lateral connections của custom FPN.	13
17	Pipeline Faster R-CNN/FPN được hiện thực trong repository.	14
18	Precision-recall/AP comparison giữa D1–D4.	15
19	Qualitative detections: đúng, false positive, false negative, small/occluded objects. .	16

Danh sách bảng

1	Đối chiếu yêu cầu bắt buộc và vị trí đáp ứng trong báo cáo.	1
2	Tổng quan dataset và khối lượng experiment.	2
3	Môi trường thực nghiệm dự kiến.	3
4	Các cấu hình E1.	4
5	Thiết lập huấn luyện E1.	5
6	Kết quả E1 trên validation/test.	5
7	Tokenization dùng trong E2.	6
8	Kế hoạch thí nghiệm E2.	7
9	Kết quả E2.	7
10	Ma trận thí nghiệm E3.	8
11	So sánh E3 với baseline E1.	9
12	Dataset/split A1 sau khi chốt.	10
13	Tám cấu hình chính A1.	11
14	Thiết lập huấn luyện A1.	11
15	Kết quả chính A1.	11
16	Bốn biến thể A2.	14
17	Thiết lập huấn luyện A2.	15
18	Kết quả chính A2.	15
19	Khung đối chiếu paper FPN và thí nghiệm nhóm.	16

1 Xác định đề bài, phạm vi và mục tiêu

Báo cáo này là **một PDF thống nhất** cho toàn bộ coursework CO5085. Cấu trúc được thiết kế trực tiếp từ ba đề E1–E3, A1 và A2. E1–E3 theo đề bắt buộc dùng một báo cáo PDF chung; theo yêu cầu tổ chức của nhóm, A1 và A2 cũng được tích hợp vào cùng master report để dữ liệu, protocol, kết quả và thảo luận nằm trong một tài liệu nhất quán.

Nguyên tắc triển khai. Mỗi kết luận trong báo cáo phải truy ngược được tới code, config và artifact chạy thực tế. Các mô hình/training loop được viết tường minh theo từng lớp/thành phần thay vì dùng trainer end-to-end. Mọi số liệu, biểu đồ và hình đánh giá đang thiếu đều giữ placeholder màu đỏ; không tự tạo kết quả.

1.1 Đối chiếu yêu cầu của đề với cấu trúc báo cáo

Bảng 1: Đối chiếu yêu cầu bắt buộc và vị trí đáp ứng trong báo cáo.

Hạng mục	Yêu cầu chính từ đề	Phần đáp ứng
E1	Softmax, MLP, CNN; training loop tự viết; validation/test; accuracy, capacity/parameters, learning curves, error cases	Mục 3
E2	MSA tự viết + bản PyTorch; ít nhất 2 tokenization; so sánh implementation/tokenizer bằng số liệu và nhận xét	Mục 4
E3	Cả LSTM và GRU; mô tả cách chuyển ảnh thành chuỗi; so sánh với MLP/CNN; bảng/biểu đồ rõ	Mục 5
A1	Dataset lớn; CNN vs Transformer; scratch/pretrained; head-only/partial/full fine-tune; accuracy/F1, time, trainable params, error analysis	Mục 6
A2	Object Detection; dataset/EDA; ít nhất 2 variants; paper method/experiments/limitations; đối chiếu paper với pipeline; results/discussion/limitations	Mục 7
Tái lập	Seed, config, log, README/lệnh chạy, code và PDF trên GitHub/GitHub Pages	Mục 2, Phụ lục A

1.2 Phạm vi thí nghiệm dự kiến

Bảng 2: Tổng quan dataset và khối lượng experiment.

Bài	Dataset / bài toán	Phạm vi dự kiến
E1–E3	Fashion-MNIST	4 classifiers; 3 tokenizers; manual/PyTorch MSA; 3 sequence representations; LSTM/GRU và E1 baselines
A1	GTSRB (đề xuất, chưa khóa)	Custom ResNet-18 vs custom ViT/DeiT-Tiny; 8 protocol chính + augmentation/seed ablation
A2	BDD100K (đề xuất, chưa khóa)	Custom Faster R-CNN pipeline; single-scale vs custom FPN; anchor và backbone fine-tuning ablations

[CHƯA CÓ HÌNH]

Luồng tổng thể của coursework: dữ liệu → EDA/split → model components
→ training → evaluation → figures/tables → report.

Hình 1: Luồng tổng thể của coursework: dữ liệu → EDA/split → model components → training → evaluation → figures/tables → report.

1.3 Quy tắc trung thực và sử dụng AI

Mọi trường chưa được xác nhận được đánh dấu **[CẦN ĐIỀN: MÀU ĐỎ]**. Synthetic smoke data chỉ dùng để kiểm tra tính chạy được của source code và **không được đưa vào phần kết quả**. Dataset test/final evaluation chỉ được dùng sau khi protocol đã được chốt bằng train/validation.

[CẦN ĐIỀN: ĐIỀN TUYÊN BỐ SỬ DỤNG AI THEO QUY ĐỊNH MÔN HỌC: phạm vi hỗ trợ, phần code/report đã được kiểm tra và người nộp hiểu/bảo vệ được nội dung.]

2 Thiết lập chung, tổ chức code và khả năng tái lập

2.1 Môi trường tính toán

Bảng 3: Môi trường thực nghiệm dự kiến.

Thành phần	Thiết lập
GPU	NVIDIA Precision A3000, 12 GB VRAM
Framework	PyTorch, torchvision, Jupyter Notebook
Seed chính	42; các cấu hình quan trọng có thể lặp thêm seed để kiểm tra độ ổn định
Thiết bị thực tế	[CẦN ĐIỀN: CHẠY scripts/check_env.py VÀ ĐIỀN CPU/RAM/CUDA/DRIVER/PYTORCH]

2.2 Quy ước code

- Không dùng `trainer.fit()` hoặc wrapper huấn luyện end-to-end.
- Training/validation/test loop được viết bằng PyTorch primitives: forward, loss, backward, optimizer step và metric accumulation.
- E2 tự viết Q/K/V, scaled dot-product attention và ghép head; `nn.MultiheadAttention` chỉ dùng làm reference.
- E3 có manual LSTM/GRU cell; `nn.LSTM/nn.GRU` dùng để đối chiếu.
- A1 tự định nghĩa ResNet-18 và ViT-Tiny theo block/layer; pretrained model chỉ cung cấp weight initialization phù hợp.
- A2 tự ghép transform, ResNet stages, custom FPN, custom anchors, RPN, proposal sampling, RoI head và Fast R-CNN predictor. Chỉ dùng operator tối ưu cấp thấp như NMS/RoIAlign từ torchvision.

2.3 Artifact và naming convention

Mỗi experiment có ID cố định, config, seed, history, thời gian, parameter count và metric. Hình cuối cùng được copy vào thư mục `assets/`; bảng/JSON/CSV phục vụ báo cáo đặt trong `report/results/`. Dataset và checkpoint lớn không commit vào GitHub.

2.4 Quy trình notebook

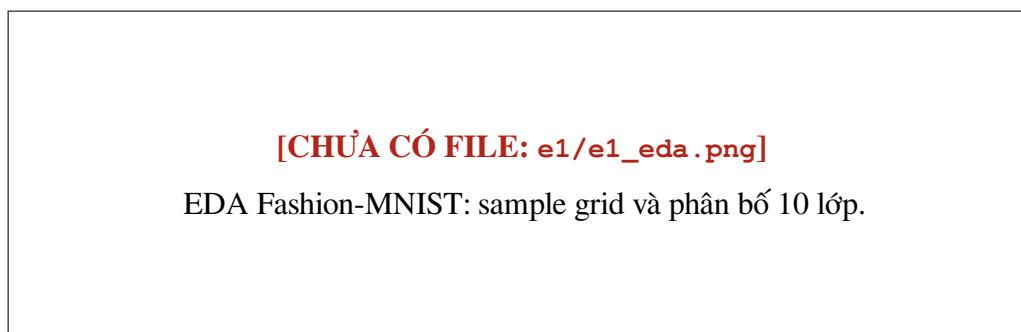
Mỗi notebook tuân theo cùng logic: **Environment** → **Data/EDA** → **Split/DataLoader** → **Model sanity** → **Train** → **Validate/Test** → **Visualize** → **Export artifacts** → **Report checklist**. A1/A2 được tách thành nhiều notebook để evaluation/plotting không buộc phải train lại model.

3 E1 - Phân loại ảnh: Softmax, MLP và CNN

3.1 Mục tiêu, dữ liệu và split

E1 sử dụng Fashion-MNIST và giữ cùng dataset/split cho E2–E3 theo yêu cầu đề. Tập train gốc sẽ được tách train/validation bằng seed cố định; test set chỉ dùng cho đánh giá cuối.

[CẦN ĐIỀN: ĐIỀN SỐ MẪU TRAIN/VAL/TEST, TỶ LỆ TÁCH VÀ SEED SAU KHI CHẠY NOTEBOOK THẬT.]



Hình 2: EDA Fashion-MNIST: sample grid và phân bố 10 lớp.

3.2 Tiền xử lý và DataLoader

Ảnh giữ cấu trúc grayscale $1 \times 28 \times 28$. Softmax/MLP flatten ảnh trong forward; CNN nhận tensor 2D trực tiếp. Báo cáo cần ghi batch size, normalization nếu dùng, số worker và lý do augmentation có/không được áp dụng.

3.3 Kiến trúc bốn cấu hình

Bảng 4: Các cấu hình E1.

ID	Mô hình	Kiến trúc chính	Câu hỏi so sánh
E1-M1	Linear Softmax	Flatten $\rightarrow$ Linear(784,10)	Baseline tuyến tính
E1-M2	MLP	$784 \rightarrow 256 \rightarrow 128 \rightarrow 10$	Giá trị của phi tuyến/depth
E1-M3	CNN-2	2 Conv blocks $\rightarrow$ classifier	Inductive bias không gian
E1-M4	CNN-3-BN	3 Conv blocks + BatchNorm + Dropout	Capacity và regularization

3.4 Training loop và protocol

Training loop phải thể hiện tường minh `optimizer.zero_grad()`, `forward`, `criterion`, `loss.backward()` và `optimizer.step()`. Tất cả mô hình dùng cùng split; hyperparameter khác biệt phải được ghi rõ thay vì ngầm thay đổi.

Bảng 5: Thiết lập huấn luyện E1.

Thành phần	Giá trị thực nghiệm
Optimizer / LR	[CẦN ĐIỀN: CHƯA CÓ DỮ LIỆU]
Batch size / epochs	[CẦN ĐIỀN: CHƯA CÓ DỮ LIỆU]
Criterion	Cross-entropy
Seed	[CẦN ĐIỀN: CHƯA CÓ DỮ LIỆU]
Model selection	[CẦN ĐIỀN: CHỌN THEO VALIDATION ACCURACY/LOSS VÀ GHI RÕ]

3.5 Kết quả định lượng

Bảng 6: Kết quả E1 trên validation/test.

Model	Params	Val Acc	Test Acc	Best epoch	Train time
E1-M1	—	—	—	—	—
E1-M2	—	—	—	—	—
E1-M3	—	—	—	—	—
E1-M4	—	—	—	—	—

[CHƯA CÓ FILE: e1/e1_learning_curves.png]

Learning curves của bốn cấu hình E1.

Hình 3: Learning curves của bốn cấu hình E1.

[CHƯA CÓ FILE: e1/e1_confusion_errors.png]

Confusion matrix và các mẫu dự đoán sai tiêu biểu của E1.

Hình 4: Confusion matrix và các mẫu dự đoán sai tiêu biểu của E1.

3.6 Phân tích lỗi và kết luận E1

[CẦN ĐIỀN: PHÂN TÍCH BẰNG EVIDENCE: lớp nào dễ nhầm, CNN cải thiện gì so với flatten, capacity có dẫn tới overfit không, tốc độ hội tụ và số tham số khác nhau thế nào.]

4 E2 - Transformer: Multi-Head Self-Attention và tokenization

4.1 Mục tiêu và nguyên tắc hiện thực

E2 dùng cùng Fashion-MNIST/split với E1. Có hai implementation attention: bản manual là implementation chính; bản `nn.MultiheadAttention` chỉ dùng đối chiếu. Không dùng `nn.TransformerEncoder` cho bản manual.

4.2 Multi-Head Self-Attention tự viết

Với token matrix $X \in \mathbb{R}^{T \times d}$, các projection được học riêng:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (1)$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \quad (2)$$

Mỗi head được tính độc lập, sau đó concat và đi qua output projection. Báo cáo cần đối chiếu tensor shape của Q, K, V , attention weights và output với implementation PyTorch.

4.3 Ba cách tokenization

Bảng 7: Tokenization dùng trong E2.

Cách	Số token	Feature/token	Ý nghĩa
Row	28	28 pixel	Mỗi hàng ảnh là một token
Patch 4×4	49	16 pixel	ViT-like non-overlapping patches
CNN-stem	49	d_{model}	CNN rút feature map 7×7 , mỗi vị trí spatial là token

[CHỮA CÓ FILE: e2/e2_tokenization.png]

Minh họa ba cách tokenization trên cùng một ảnh Fashion-MNIST.

Hình 5: Minh họa ba cách tokenization trên cùng một ảnh Fashion-MNIST.

4.4 Classifier và ma trận thí nghiệm

Classifier gồm tokenizer, CLS token, hai encoder block nông, LayerNorm và classification head. Để tăng độ sâu so sánh nhưng vẫn bám đề, thực hiện sáu run chính ($2 \text{ MSA} \times 3 \text{ tokenizers}$) và ba run ablation số head 2/4/8 trên tokenizer tốt nhất.

Bảng 8: Kế hoạch thí nghiệm E2.

Nhóm	MSA	Tokenizer	Heads	Mục đích
E2-A1–A3	Manual	Row/Patch/CNN-stem	4	So sánh tokenization
E2-B1–B3	PyTorch	Row/Patch/CNN-stem	4	Manual vs reference
E2-C1–C3	Manual	Best tokenizer	2/4/8	Ablation số head

4.5 Kết quả và attention visualization

Bảng 9: Kết quả E2.

ID	MSA	Tokenizer	Heads	Val Acc	Test Acc	Time/epoch
CHƯA CÓ KẾT QUẢ - SẼ ĐƯỢC NOTEBOOK XUẤT TỪ RUN THẬT						

[CHƯA CÓ FILE: e2/e2_comparison.png]

So sánh accuracy và thời gian giữa manual/PyTorch MSA và các tokenizer.

Hình 6: So sánh accuracy và thời gian giữa manual/PyTorch MSA và các tokenizer.

[CHƯA CÓ FILE: e2/e2_attention_maps.png]

Attention heatmap của một số mẫu Fashion-MNIST.

Hình 7: Attention heatmap của một số mẫu Fashion-MNIST.

4.6 Thảo luận E2

[CẦN ĐIỀN: PHÂN TÍCH: sequence length ảnh hưởng compute ra sao; tokenizer nào tốt; manual và PyTorch có xu hướng tương đồng không; số head giúp/hại thế nào; attention map có hợp lý về mặt thị giác không.]

5 E3 - LSTM/GRU trên chuỗi ảnh

5.1 Mục tiêu và các cách biến ảnh thành chuỗi

E3 tiếp tục dùng Fashion-MNIST. Thay vì chỉ chọn một representation, báo cáo chuẩn bị ba cách để tạo đủ evidence cho comparison:

1. **Rows:** $T = 28$, mỗi timestep có 28 feature.
2. **Columns:** $T = 28$, mỗi timestep có 28 feature.
3. **Patch** 4×4 : $T = 49$, mỗi timestep có 16 feature.

5.2 Manual LSTM và GRU

Repository có manual cell theo phương trình cổng để kiểm chứng kiến thức. `nn.LSTM/nn.GRU` vẫn được giữ làm reference vì đề cho phép, nhưng không thay thế phần manual.

LSTM. Báo cáo cần trình bày input, forget, candidate và output gates; chỉ rõ cách cập nhật c_t và h_t .

GRU. Báo cáo cần trình bày update/reset gates và hidden candidate; giải thích vì sao GRU có ít state hơn LSTM.

[CHƯA CÓ FILE: `e3/e3_recurrent_cells.png`]

Sơ đồ manual LSTM/GRU cell và luồng ảnh $\rightarrow$ sequence $\rightarrow$ classifier.

Hình 8: Sơ đồ manual LSTM/GRU cell và luồng ảnh $\rightarrow$ sequence $\rightarrow$ classifier.

5.3 Ma trận thí nghiệm

Bảng 10: Ma trận thí nghiệm E3.

ID	Recurrent model	Representation	Mục đích
E3-L-R/E3-G-R	LSTM / GRU	Rows	Baseline sequence
E3-L-C/E3-G-C	LSTM / GRU	Columns	Sensitivity theo hướng quét
E3-L-P/E3-G-P	LSTM / GRU	Patch 4×4	Local block sequence
E1 baseline	MLP / best CNN	2D image	So sánh non-recurrent

5.4 Kết quả

Bảng 11: So sánh E3 với baseline E1.

Model	Representation	Params	Val Acc	Test Acc	Time/epoch	Best epoch
LSTM	Row	—	—	—	—	—
GRU	Row	—	—	—	—	—
LSTM	Column	—	—	—	—	—
GRU	Column	—	—	—	—	—
LSTM	Patch	—	—	—	—	—
GRU	Patch	—	—	—	—	—
Best E1 CNN	2D image	—	—	—	—	—

[CHƯA CÓ FILE: e3/e3_learning_comparison.png]

Learning curves và so sánh E3 với MLP/CNN baseline.

Hình 9: Learning curves và so sánh E3 với MLP/CNN baseline.

5.5 Thảo luận E3

[CẦN ĐIỀN: PHÂN TÍCH LSTM VS GRU, ROW VS COLUMN VS PATCH, PARAMETER/TIME TRADE-OFF VÀ VÌ SAO MÔ HÌNH 2D CNN CÓ THỂ CÓ LỢI THẾ TRÊN ẢNH.]

6 A1 - CNN vs Transformer trên tập ảnh lớn

6.1 Bài toán, dataset và A1.1 protocol

Dataset đề xuất hiện tại là **GTSRB** cho bài toán traffic-sign recognition. Trước A1.1 cần chốt và ghi rõ nguồn, license, số lớp, số ảnh, official split/subset và lý do chọn. Dataset phải lớn/khó hơn E1 và không dùng Fashion-MNIST/MNIST/CIFAR-10 làm tập chính.

[CẦN ĐIỀN: XÁC NHẬN CUỐI CÙNG DATASET A1 TRƯỚC 26/10/2026.]

[CHƯA CÓ FILE: a1/a1_eda.png]

EDA A1: class distribution, image-size distribution và sample grid.

Hình 10: EDA A1: class distribution, image-size distribution và sample grid.

6.2 Split, DataLoader và augmentation

Ảnh được resize về 224×224 . Training dùng augmentation; validation/test dùng deterministic transform. Validation được dùng chọn protocol/hyperparameter; test chỉ dùng sau khi chốt model. Báo cáo phải ghi rõ normalize statistics, batch size, số worker và mọi subset nếu GPU giới hạn.

Bảng 12: Dataset/split A1 sau khi chốt.

Split	Số ảnh	Số lớp	Ghi chú
Train	–	–	[CẦN ĐIỀN: OFFICIAL/SUBSET?]
Validation	–	–	[CẦN ĐIỀN: CÁCH TÁCH]
Test	–	–	Giữ cho final evaluation

6.3 CNN: custom ResNet-18

ResNet-18 được định nghĩa bằng `BasicBlock`, stem, four residual stages, global average pooling và classifier. Bản pretrained chỉ chuyển ImageNet weights vào các layer tương thích; head mới vẫn được train theo dataset A1.

6.4 Transformer: custom ViT/DeiT-Tiny

Transformer được tách thành PatchEmbedding, CLS/position embeddings, manual self-attention, MLP và encoder blocks. Bản pretrained dùng matching weights từ model tham chiếu; architecture forward và training loop của repository vẫn tường minh.

[CHƯA CÓ FILE: a1/a1_architecture.png]

Sơ đồ ResNet-18 và ViT/DeiT-Tiny dùng trong A1.

Hình 11: Sơ đồ ResNet-18 và ViT/DeiT-Tiny dùng trong A1.

6.5 Protocol scratch, pretrained và freezing

Bảng 13: Tám cấu hình chính A1.

ID	Họ mô hình	Khởi tạo	Chế độ train
C1	ResNet-18	Random	From scratch
C2	ResNet-18	ImageNet	Freeze backbone / train head
C3	ResNet-18	ImageNet	Partial fine-tune
C4	ResNet-18	ImageNet	Full fine-tune
T1	ViT/DeiT-Tiny	Random	From scratch
T2	ViT/DeiT-Tiny	ImageNet	Freeze encoder / train head
T3	ViT/DeiT-Tiny	ImageNet	Partial fine-tune
T4	ViT/DeiT-Tiny	ImageNet	Full fine-tune

Ngoài tám cấu hình bắt buộc/mở rộng trên, best CNN và best Transformer có thể chạy augmentation ablation và thêm seed để đánh giá độ ổn định. Phần này chỉ được thực hiện sau khi protocol chính chạy ổn định.

6.6 Thiết lập huấn luyện và chi phí

Bảng 14: Thiết lập huấn luyện A1.

Thành phần	Giá trị
Optimizer/LR/scheduler	[CẦN ĐIỀN: CHƯA CÓ DỮ LIỆU]
Epoch / early stopping	[CẦN ĐIỀN: CHƯA CÓ DỮ LIỆU]
Batch size / image size	[CẦN ĐIỀN: CHƯA CÓ DỮ LIỆU]
Model-selection metric	[CẦN ĐIỀN: VALIDATION ACCURACY HOẶC MACRO-F1, CHỐT SAU EDA]
Compute log	total params, trainable params, epoch time, total time, peak VRAM

6.7 Kết quả so sánh

Bảng 15: Kết quả chính A1.

ID	Acc	Macro-F1	Trainable params	Time	Peak VRAM	Seed
C1	—	—	—	—	—	42
C2	—	—	—	—	—	42
C3	—	—	—	—	—	42
C4	—	—	—	—	—	42
T1	—	—	—	—	—	42
T2	—	—	—	—	—	42
T3	—	—	—	—	—	42
T4	—	—	—	—	—	42

[CHƯA CÓ FILE: a1/a1_learning_curves.png]

Learning curves của các cấu hình A1 tiêu biểu.

Hình 12: Learning curves của các cấu hình A1 tiêu biểu.

[CHƯA CÓ FILE: a1/a1_confusion_errors.png]

Confusion matrix và error gallery của best CNN/Transformer.

Hình 13: Confusion matrix và error gallery của best CNN/Transformer.

[CHƯA CÓ FILE: a1/a1_cost_tradeoff.png]

Accuracy/F1 so với trainable parameters, training time và peak VRAM.

Hình 14: Accuracy/F1 so với trainable parameters, training time và peak VRAM.

6.8 Thảo luận A1

[CẦN ĐIỀN: TRẢ LỜI BẰNG SỐ LIỆU: pretraining giúp bao nhiêu; head-only/partial/full khác nhau thế nào; ViT có cần pretraining hơn CNN không; khi nào freeze có lợi; lỗi theo lớp; compute trade-off.]

7 A2 - Object Detection và phân tích paper

7.1 Bài toán, dataset và A2.1 protocol

Chủ đề đã chọn là **Object Detection**. Dataset đề xuất hiện tại là BDD100K detection vì có road scenes, nhiều class và object ở nhiều scale. Trước A2.1 cần chốt dataset/subset, class mapping, split, metric và paper citation.

[CẦN ĐIỀN: XÁC NHẬN DATASET A2, QUY MÔ SUBSET VÀ PAPER CUỐI CÙNG TRƯỚC 02/11/2026.]

[CHƯA CÓ FILE: a2/a2_eda.png]

EDA A2: số box theo lớp, số object/ảnh, phân bố diện tích/aspect ratio và sample annotations.

Hình 15: EDA A2: số box theo lớp, số object/ảnh, phân bố diện tích/aspect ratio và sample annotations.

7.2 Dataset, DataLoader và augmentation

Mỗi sample trả về ảnh và target gồm boxes, labels, area, image id và iscrowd. Ảnh được resize về một canvas cố định cho detector; bounding boxes được scale tương ứng. Horizontal flip hoặc augmentation hình học chỉ được dùng khi code cập nhật box chính xác. Validation/test không dùng augmentation ngẫu nhiên.

7.3 Paper chính: Feature Pyramid Networks for Object Detection

Paper chính dự kiến là Lin et al., *Feature Pyramid Networks for Object Detection* [5]; Faster R-CNN [4] là detector nền. Phần paper phải viết lại bằng lời của nhóm, không chép abstract, và gồm đủ method, experiments, limitations và liên hệ với pipeline.

7.3.1 Ý tưởng, kiến trúc và giả thiết

FPN dùng top-down pathway và lateral connections để tạo pyramid feature có semantic mạnh ở nhiều resolution. Backbone tạo C_2, C_3, C_4, C_5 ; mỗi level được chiếu bằng convolution 1×1 , cộng với feature cấp cao được upsample, rồi convolution 3×3 tạo P_2, P_3, P_4, P_5 .

[CHƯA CÓ FILE: a2/a2_fpn_architecture.png]

Top-down pathway và lateral connections của custom FPN.

Hình 16: Top-down pathway và lateral connections của custom FPN.

7.3.2 Thí nghiệm của paper

[CẦN ĐIỀN: ĐỌC PAPER CHÍNH THỨC VÀ ĐIỀN: DATASET, BACKBONE, DETECTOR, METRIC, ABLATION, KẾT LUẬN; CITE TỪNG CLAIM QUAN TRỌNG.]

7.3.3 Hạn chế của paper và khả năng tái hiện

Báo cáo phải phân biệt rõ *full benchmark reproduction* với *controlled/partial reproduction*. Nếu nhóm dùng BDD100K/subset, schedule khác và pretrained backbone thì không được tuyên bố tái hiện đúng số liệu COCO của paper; mục tiêu là kiểm tra lại contribution FPN trong protocol có kiểm soát.

7.4 Pipeline Faster R-CNN tự ghép theo từng thành phần

Repository không gọi `torchvision.models.detection.FasterRCNN` và không gọi `fasterrcnn_resnet50`. Pipeline được viết thành các module độc lập:

1. fixed-size detection transform và box scaling;
2. ResNet stages và single-scale/FPN backbone;
3. custom anchor generator;
4. custom RPN head, anchor matching, sampling, box regression và proposal generation;
5. RoIAlign là operator cấp thấp từ torchvision;
6. custom RoI sampling, two-FC box head và class-specific box predictor;
7. custom training loss assembly và inference post-processing;
8. NMS là operator tối ưu cấp thấp từ torchvision.

[CHỮA CÓ FILE: a2/a2_pipeline.png]

Pipeline Faster R-CNN/FPN được hiện thực trong repository.

Hình 17: Pipeline Faster R-CNN/FPN được hiện thực trong repository.

7.5 Ma trận thí nghiệm A2

Bảng 16: Bốn biến thể A2.

ID	Detector	Biến thay đổi	Câu hỏi
D1	Faster R-CNN single-scale	Chỉ C5 projection, không FPN	Baseline detection
D2	Faster R-CNN + FPN	Custom P2–P5	FPN cải thiện multi-scale detection thế nào?
D3	FPN + small anchors	Anchor scales nhỏ hơn	Có tăng AP của small objects không?
D4	FPN + deeper fine-tune	Mở thêm ResNet stages	Accuracy/compute trade-off thế nào?

7.6 Training protocol

Backbone có thể khởi tạo ImageNet pretrained; detector heads/FPN được train trên dataset A2. Không bắt buộc train toàn bộ backbone từ random initialization. Để D1–D2 là controlled comparison, cần giữ giống nhau split, augmentation, image size, optimizer, schedule và backbone initialization; biến chính là FPN.

Bảng 17: Thiết lập huấn luyện A2.

Thành phần	Giá trị
Train/val/subset	[CẦN ĐIỀN: CHƯA CÓ DỮ LIỆU]
Image size	[CẦN ĐIỀN: CHƯA CÓ DỮ LIỆU]
Backbone init / trainable stages	[CẦN ĐIỀN: CHƯA CÓ DỮ LIỆU]
Optimizer / LR / schedule	[CẦN ĐIỀN: CHƯA CÓ DỮ LIỆU]
RPN anchors / NMS thresholds	[CẦN ĐIỀN: CHƯA CÓ DỮ LIỆU]
Model selection	validation mAP

7.7 Kết quả định lượng

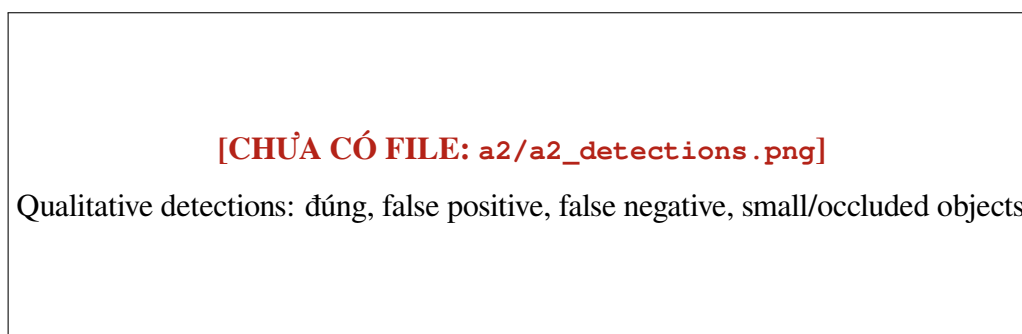
Bảng 18: Kết quả chính A2.

ID	mAP@0.5	mAP@0.5:0.95	AP_S	AP_M	AP_L	Latency
D1	–	–	–	–	–	–
D2	–	–	–	–	–	–
D3	–	–	–	–	–	–
D4	–	–	–	–	–	–

[CHƯA CÓ FILE: a2/a2_pr_ap.png]

Precision-recall/AP comparison giữa D1–D4.

Hình 18: Precision-recall/AP comparison giữa D1–D4.



Hình 19: Qualitative detections: đúng, false positive, false negative, small/occluded objects.

7.8 Failure-case analysis

Phân nhóm lỗi theo small objects, occlusion, dense traffic, day/night, weather và class confusion. Các ví dụ phải được lấy từ output thật và đi kèm confidence/ground-truth để tránh cherry-picking.

7.9 Đối chiếu paper với pipeline của nhóm

Bảng 19: Khung đối chiếu paper FPN và thí nghiệm nhóm.

Yếu tố	Paper	Pipeline nhóm
Dataset	[CẦN ĐIỀN: ĐIỀN]	BDD100K (đề xuất, chưa khóa)
Backbone/init	[CẦN ĐIỀN: ĐIỀN]	[CẦN ĐIỀN: ĐIỀN CONFIG THẬT]
Detector/FPN	[CẦN ĐIỀN: ĐIỀN]	Custom Faster R-CNN + custom FPN
Training schedule	[CẦN ĐIỀN: ĐIỀN]	[CẦN ĐIỀN: ĐIỀN]
Metric	[CẦN ĐIỀN: ĐIỀN]	mAP@0.5, mAP@0.5:0.95, AP by size
Kết luận	[CẦN ĐIỀN: ĐIỀN]	[CẦN ĐIỀN: ĐIỀN SAU D1-D4]

7.10 Hạn chế và kết luận A2

[CẦN ĐIỀN: THẢO LUẬN GIỚI HẠN GPU 12GB, SUBSET NẾU CÓ, PRETRAINED BACKBONE, CLASS IMBALANCE, SMALL OBJECTS, WEATHER/TIME-OF-DAY DOMAIN SHIFT, VÀ MỨC ĐỘ REPRODUCTION THỰC SỰ.]

8 Kết luận và hướng mở rộng

Phần kết luận cuối cùng phải tổng hợp theo evidence thay vì chỉ liệt kê accuracy. Cần nổi ba tầng kiến thức của môn: mô hình cơ bản (E1), attention/sequence modeling (E2–E3), transfer learning và kiến trúc quy mô lớn (A1), sau đó multi-scale object detection và paper reproduction (A2).

[CẦN ĐIỀN: ĐIỀN KẾT LUẬN SAU KHI HOÀN THÀNH TẤT CẢ THÍ NGHIỆM.]

Các hướng mở rộng phù hợp với đề gồm Grad-CAM/attention visualization, demo inference, cross-dataset evaluation hoặc ablation bổ sung nếu còn tài nguyên.

A Phụ lục: lệnh chạy và tái lập

A.1 Cài đặt

```
git clone https://github.com/dangtai111325/CO5085.git
cd CO5085
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python scripts/check_env.py
python scripts/smoke_test.py
```

A.2 Build báo cáo

```
bash scripts/build_report.sh
```

A.3 Artifact cần giữ để tái lập

- seed, config YAML và package versions;
- CSV/JSON history của từng run;
- checkpoint tốt nhất (lưu local/release nếu kích thước phù hợp, không bắt buộc commit Git);
- figures/tables đã dùng trong báo cáo;
- commit/tag tương ứng với từng deadline.

Tài liệu tham khảo

References

- [1] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” CVPR, 2016.
- [2] A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” ICLR, 2021.
- [3] H. Touvron et al., “Training Data-Efficient Image Transformers and Distillation through Attention,” ICML, 2021.
- [4] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” NeurIPS, 2015.
- [5] T.-Y. Lin et al., “Feature Pyramid Networks for Object Detection,” CVPR, 2017.
- [6] F. Yu et al., “BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning,” CVPR, 2020.